Sub-Agents: Consolidated Prompts

A learning prototype: **1 orchestrator + exactly 2 subagents** (Booking.com, Airbnb) that finds the top 3 stays per source, ranks them by value in code, and shows a ⭐ Best pick.

Contents

- **Part 1. Quick version:** one prompt per step, with every later follow-up and bug fix already folded in.
- **Part 2. Full version:** every prompt verbatim, in the order it was sent, plus the lessons learned.

Run `/step-check N` after each step. Design lineage: Figma design (SearchHotel, prototype-2) → written spec `docs/design-spec-v3.md` → the UI built here.

Part 1. Quick version (one prompt per step)

Prompt 0: GitHub repo

```
STEP 0 ONLY, no code. Folder D:\Projects\Sub-Agents. Protected (never modify/print/commit):
CLAUDE.md, docs/, .claude/, .env.local.
Empty PUBLIC repo exists: https://github.com/rajanjinesh/sub-agents.git
1. Create .gitignore: node_modules, .next, .vercel, .env*, !.env.example, .DS_Store
2. git init, default branch main. 3. Show git status; confirm .env.local is NOT listed.
4. Scan files to commit for API keys; stop if found.
5. Commit CLAUDE.md, docs/, .claude/, .gitignore as "chore: project docs".
6. Add origin, push -u origin main (never ask me for a password/token in chat).
7. Show git log --oneline and git remote -v. Stop.
```

Prompt 1: Setup + Search page

```
STEP 1 ONLY (CLAUDE.md §13). No APIs/agents. Read CLAUDE.md and docs/design-spec-v3.md first. Name
is "Sub-Agents" everywhere, never "Hotel Finder".
- Scaffold Next.js (App Router, TS, Tailwind, ESLint, src/, Inter via next/font); if the folder
isn't empty, scaffold in a temp folder and move files in. Keep .gitignore; add .env.example
(ANTHROPIC_API_KEY, ANTHROPIC_MODEL, HASDATA_API_KEY, USE_FIXTURES, empty).
- Design tokens from the spec (colors incl. #eff6ff/#f3f4f6, 8px radius, 999px pill, Inter 400-700,
550px column, 21px top padding, 16px side padding <600px, inputs 36px).
- Search page exactly per spec: title, subtitle "Compare stays on Booking.com and Airbnb",
destination ("e.g. Goa", "Please enter a destination"), check-in default today "29 Oct 2026 (Today)"
not in past, check-out default tomorrow ("Check-out must be after check-in", #ef4444), live "{n}
night(s)", adults (min 1), children (min 0), one 0-17 age dropdown per child (required), Search /
"Searching…" (disabled, opacity 0.6).
- Valid submit → /results?destination=&checkIn=&checkOut=&adults=&children=&childAges=1,5
(placeholder page).
- Files: src/lib/types.ts, format.ts (formatINR ₹4,200 Indian grouping, formatDate, nights),
validate.ts (shared client/server), small components.
- Show git status, no env file staged. npm run build passes, commit, push (triggers first Vercel
deploy). Summarize ambiguities. Stop.
```

Prompt 2: HasData data layer (follow-up decisions and fixes included)

```
STEP 2 ONLY. No Anthropic SDK/agents/UI. Re-read CLAUDE.md §3, 4, 6.
CREDIT RULE: exactly ONE live call per source (Goa, check-in +14 days, 1 night, 2 adults, 1 child
age 5); save raw to fixtures/booking-raw.json and airbnb-raw.json. Everything else runs on fixtures.
- Clients with x-api-key (server-side), clear errors for missing key / 401 / non-200 / 15s timeout;
check HasData docs for param names.
  Booking: GET /scrape/booking/search: keyword, checkInDate, checkOutDate, rooms=1, adults,
children, childrenAges (comma list, only if children>0), currency=inr (lowercase).
```

```
  Airbnb: GET /scrape/airbnb/listing: location, checkIn, checkOut, adults, children (count only;
OMIT children when 0, HasData returns 422).
- Booking: total = pricePerStayParsed + excludedChargesParsed; rating = reviews.score (/10), not the
star class.
- Airbnb: prices are USD, no currency param. Use ONE constant USD_TO_INR in src/lib/config.ts (tell
me the value); total = nights × nightly from the "N nights x $X" breakdown; long stays (28+ nights)
use "Average monthly price" ÷ 30 × nights (nights from the echoed search URL); rating ×2;
feesMayBeExtra = true always.
- normalizeBooking/normalizeAirbnb → Stay[] (exact CLAUDE.md §6 shape). rules.ts applyRules: rating
≥7.0, unrated out, no hostel/dorm/shared room, INR total incl. taxes/fees, 1 room, unparseable price
skipped, 3 cheapest.
- Add vitest (@types/node ^24) with tests on fixtures. Write docs/hasdata-fields.md.
- GET /api/dev/stays?source=booking|airbnb from fixtures (&live=1 for real; 404 in production).
- Commit CLAUDE.md updates too. Summarize raw count / passed rules / ambiguities. Stop.
```

Prompt 3: The two subagents

```
STEP 3 ONLY. No orchestrator/UI. Re-read CLAUDE.md §3.
- Install @anthropic-ai/sdk. src/lib/anthropic.ts is the ONLY place the client is created
(ANTHROPIC_API_KEY, ANTHROPIC_MODEL; clear error if missing; server-only).
- USE_FIXTURES switch (missing = true locally): tools return Step 2 fixtures. Name the function
fixturesEnabled().
- src/lib/agents/bookingAgent.ts and airbnbAgent.ts: runBookingAgent/runAirbnbAgent(input). Each is
its own Claude call with its own short system prompt ("You find stays on {source}. Use your tool
once. Don't rank or filter, just report what the tool returned.") and exactly ONE tool
(search_booking / search_airbnb).
- Loop: model calls tool → our code fetch→normalize→applyRules → tool_result → model replies one
line. Stays come from tool code, never the model's text. Max 3 turns; errors become a failed
SourceResult.
- After the tool call, compare the model's params with the user's SearchInput; if anything differs,
use the SearchInput values and add a "params corrected" warning to the trace. Add a test.
- Return { stays, ms, trace } (agent started / tool called / tool returned / finished).
- GET /api/dev/agent?source=… (404 in production). Test with mocked Anthropic client. Commit, push,
leave dev server running. Stop.
```

Prompt 4: Orchestrator, rank(), streaming

```
STEP 4 ONLY. No UI changes. Re-read CLAUDE.md §3, 5, 6, 7.
- src/lib/rank.ts pure rank(): sort by totalPrice asc (tie: higher rating first); repeatedly swap
adjacent A,B when B ≤10% pricier AND rated ≥1.0 higher. Tests: no swaps, one swap, chain, ties,
single, empty, exactly-10% and exactly-1.0 boundaries.
- orchestrator.ts runOrchestrator(input, emit): own Claude call ("You coordinate two subagents. Call
both tools in the same turn.") with exactly 2 tools call_booking_agent, call_airbnb_agent. Run both
subagents IN PARALLEL with Promise.allSettled; if the model calls only one, call the other in code
and warn. Subagent input always = user's SearchInput. Merge all traces in time order, tagged by
agent.
```

```
- Code combines → rank() → max 6. The model only writes the Best pick reason (≤90 chars, from #1 and #2). Be honest about the trade-off: cheaper but lower-rated → say so ("₹663 cheaper than the next option, rated 9.5 vs 10."); pricier but higher-rated → say that. Never call a lower rating a plus. Give the model code-computed facts; empty/too-long → code-written fallback. 1 stay → reason null; 0 → none. Test each case.
- POST /api/search (SSE): validate with validateSearchInput (400); server allows check-in from 1 day before its UTC date (client stays strict; test it); fail fast if ANTHROPIC key missing; events source_started, source_done{count}, source_failed{error}, ranking_started, result; maxDuration=60 (no runtime export, Next 16 cacheComponents); one failure → partial.
- GET /api/dev/search?fail=booking|airbnb returns {events, result, lines} (404 in production).
- Commit, push, leave dev server running. Stop.
```

Prompt 5: Loading, Results and all states

```
STEP 5 ONLY. Re-read design-spec Screens 2-4; match copy, sizes, colors exactly.
- Search submit → button "Searching…" (disabled, 0.6) → /results; /results POSTs /api/search and reads SSE with fetch+ReadableStream. Read URL params in the BROWSER (useSearchParams inside Suspense), not on the server. Start the search once (Strict Mode).
- Loading: "Finding your stays…" + summary; rows "Searching Booking.com…/Airbnb…" with 16px #2563eb spinners → "✓ {Source}: {n} stays found" or muted "✗ {Source}: unavailable"; then "Picking the best stay…".
- Results: "Best stays for your trip", summary "{destination} · {dates} · {n} night(s) · {guests}", sub-line "{n} stays compared from …" naming ONLY the sources that succeeded, "← New search" (keeps inputs; Search page prefills from the URL; reset "Searching…" when leaving).
- Cards: source pill, name, "₹4,200 total stay", rating with one decimal ("9.5 / 10 rating", 10 → "10 / 10"), "View on Booking.com/Airbnb" (new tab), "Fees may be extra". Best pick card: 2px #2563eb border, #eff6ff fill, "⭐ Best pick" pill, reason line.
- States with exact copy: Partial (muted note), Empty, Error, 1-5 stays.
- ?debug=1 shows an "Agent activity" timeline (hidden by default).
- Test each state (dev-only hooks DEBUG_FAIL_SOURCE/DEBUG_EMPTY_SOURCE), desktop and <600px. Commit, push, leave dev server running at localhost:3000. Stop.
```

Prompt 6: Go live

```
STEP 6 ONLY.
- Pre-ship: build, lint, tests pass. Search the whole git history for env files and the real key values (compare by value, never print them); stop if found.
- README.md: what it is, run locally, env var names only, how the orchestrator invokes the 2 subagents in parallel, where code (rules, rank) takes over from the LLM, the flow diagram from CLAUDE.md, fixture vs live mode and credits per live search (Booking 10 + Airbnb 5 ≈ 15).
- Commit and push main. I confirm USE_FIXTURES=false and the keys in Vercel (never show values).
- Confirm dev routes are 404 in production and /api/search doesn't time out.
- Run only ONE live search on https://sub-agents-three.vercel.app; tell me how long it took and the credits used. Give me the live URL, repo URL and credits. Stop.
```

Optional Prompt 0A (suggested, not from the chat): Figma → spec

```
Read my Figma design <Figma link> with the Figma connector. Write docs/design-spec-v3.md: global tokens (colors, fonts, radius, spacing), each screen with exact copy, sizes and states. Don't build any code.
```

Part 2. Full version

The rest of this document lists every prompt verbatim, in the order it was sent.

A learning prototype: **1 orchestrator agent + exactly 2 subagents** (Booking.com, Airbnb) that finds the top 3 stays per source, ranks them by value in code, and shows a ⭐ Best pick.

Everything below was taken from the full chat. Step prompts are copied **verbatim** from `docs/build-prompts.md`; the messages you typed in chat are included in the order you sent them (typos tidied, meaning unchanged). Nothing is paraphrased inside a code block.

> **About Figma:** the UI is based on the Figma design. The lineage is **Figma design ("SearchHotel", prototype-2) → written spec `docs/design-spec-v3.md` → the UI built in this project** (`docs/design-spec-v3.md`, line 3: "Based on prototype-2 design spec (Figma 'SearchHotel')"). The step prompts in this chat never open Figma themselves: they say "re-read docs/design-spec-v3.md and match copy, sizes and colors exactly", so the Figma design reached the code through that spec file. The prompt or process that turned the Figma design into the spec happened **before** this project and is not in this chat, so it is not listed here. The stays data (top 3 per source) comes from the **HasData** REST API.

How the pieces fit

Piece	File	Role
Product spec (PRD)	`CLAUDE.md`	Wins on product behaviour
UI spec	`docs/design-spec-v3.md`	Wins on UI, copy, tokens, states
Step prompts	`docs/build-prompts.md`	The prompts for Steps 0–6
Review skill	`.claude/skills/step-check/SKILL.md`	The `/step-check` quality gate
Secrets	`.env.local` (never committed)	`ANTHROPIC_API_KEY`, `ANTHROPIC_MODEL`, `HASDATA_API_KEY`, `USE_FIXTURES`

Agent flow (from CLAUDE.md):

```
User → POST /api/search (streams progress)
          |
     Orchestrator agent
       ├── call_booking_agent ──► Booking subagent ──► search_booking ──► HasData Booking API
       └── call_airbnb_agent  ──► Airbnb subagent  ──► search_airbnb  ──► HasData Airbnb API
          |
     rank() in code → orchestrator LLM writes Best pick reason → results
```

Part A. Foundation (done before the chat started)

You prepared these outside the chat. Re-create them first if you start over:

1. `CLAUDE.md` (the PRD) and `docs/design-spec-v3.md` (the UI spec) in `D:\Projects\Sub-Agents`.
2. `docs/build-prompts.md` (the step prompts below) and the `step-check` skill moved into `.claude/skills/step-check/`.
3. `.env.local` with your four variables (`USE_FIXTURES=true` locally).
4. An **empty public** GitHub repo `https://github.com/rajanjinesh/sub-agents.git` (no README, .gitignore or licence).
5. A Vercel project `sub-agents` linked to that repo, live URL `https://sub-agents-three.vercel.app`, with the four env vars added (`USE_FIXTURES=false` on Vercel). Its first deploy fails until Step 1 pushes an app.

How to use each step prompt: paste only what is inside the code block, let it finish, then run `/step-check N`.

Part B. The prompts, in order

1. Step 0: GitHub repo

Sets up git and pushes the docs. No code.

```
## Context
This folder is the "Sub-Agents" project (D:\Projects\Sub-Agents).
This is STEP 0 ONLY: set up git and connect it to my GitHub repo. Don't scaffold the app
or write any code.

I've already created an EMPTY PUBLIC repo on GitHub: https://github.com/rajanjinesh/sub-agents.git

Protected (DO NOT modify or delete): CLAUDE.md, docs/, .claude/, .env.local.
Never print, log or commit .env.local. The repo is public, so anything committed is visible
to everyone.

## Tasks
1. Create .gitignore with: node_modules, .next, .vercel, .env*, !.env.example, .DS_Store
2. git init with default branch "main".
3. Run `git status` and show me. Confirm .env.local is NOT listed.
4. Scan the files to be committed for anything that looks like an API key. If found, stop.
5. Commit CLAUDE.md, docs/, .claude/ and .gitignore with the message "chore: project docs".
6. git remote add origin https://github.com/rajanjinesh/sub-agents.git, then git push -u origin
main.
   If the push asks me to sign in, tell me what to do. Never ask me for a password or token in chat.
7. Show me `git log --oneline` and `git remote -v`.

## Done when
- The public repo "sub-agents" on my GitHub has CLAUDE.md, docs/, .claude/skills/step-check
```

```
  and .gitignore, and no env file.
- Stop; don't start Step 1.
```

2. Review gate

```
/step-check 0
```

3. Step 1: Setup + Search page

Next.js scaffold, design tokens, the search form with validation, `/results` placeholder, first Vercel deploy.

```
## Context
This folder is the "Sub-Agents" project (D:\Projects\Sub-Agents), a learning prototype:
1 orchestrator agent + exactly 2 subagents (Booking.com, Airbnb).
Use "Sub-Agents" everywhere: package name "sub-agents", on-screen title "Sub-Agents",
GitHub repo https://github.com/rajanjinesh/sub-agents (public, connected to Vercel).
Never use the name "Hotel Finder".

Already in this folder (DO NOT modify, overwrite or delete):
- CLAUDE.md: the PRD. Read it fully first and follow it.
- docs/design-spec-v3.md: the UI spec. Read it fully first.
- .env.local with my real API keys. Never print, log, commit or overwrite it.
- .claude/skills/step-check: my review skill. Don't touch it.

This is STEP 1 ONLY (see CLAUDE.md section 13): project setup + Search page.
No API calls, no agents, no HasData, no Anthropic SDK yet.

## Tasks

### 1. Scaffold Next.js
- Next.js App Router, TypeScript, Tailwind, ESLint, src/ dir, Inter via next/font.
- create-next-app may refuse because the folder isn't empty. If so, scaffold into a temp
  folder and move the generated files in. Keep CLAUDE.md, docs/, .claude/ and .env.local untouched.
- Git and the GitHub repo already exist (Step 0). Keep the existing .gitignore and add any
  Next.js entries it's missing.
- Create .env.example with these variable names and empty values:
  ANTHROPIC_API_KEY, ANTHROPIC_MODEL, HASDATA_API_KEY, USE_FIXTURES
- Before committing, show me `git status` and confirm no env file is staged.

### 2. Design tokens
From docs/design-spec-v3.md: all colors (including #eff6ff and #f3f4f6), 8px radius,
999px pill radius, Inter 400/500/600/700, 550px centered column, 21px top padding,
16px side padding under 600px, inputs 36px high with 8px 12px padding.
Put them in Tailwind theme / CSS variables so later steps reuse them.

### 3. Search page (Screen 1), exactly per the spec
- Title "Sub-Agents" + subtitle "Compare stays on Booking.com and Airbnb".
```

```
- Destination field, placeholder "e.g. Goa", error "Please enter a destination".
- Check-in defaults to today, displayed like "29 Oct 2026 (Today)". Can't be in the past.
- Check-out defaults to tomorrow. Error "Check-out must be after check-in" (#ef4444).
- "{n} night(s)" updates live.
- Adults (default 1, min 1), Children (default 0, min 0).
- One age dropdown (0–17) per child, added and removed with the count. Required.
- Search button. "Searching…" + disabled (opacity 0.6) state ready for later.
- On valid submit, go to /results?destination=&checkIn=&checkOut=&adults=&children=&childAges=1,5
- /results is a placeholder for now. Just show the parsed params.

### 4. Files to create now
src/lib/types.ts    SearchInput (from CLAUDE.md section 6)
src/lib/format.ts   formatINR (₹4,200, Indian grouping), formatDate, nights()
src/lib/validate.ts validateSearchInput(), shared by client now and server later
src/components/     small form field components

## Done when
- npm run dev works. The Search page matches the spec at desktop and mobile (<600px) widths.
- All validation errors show correctly. Submitting lands on /results with the right params.
- No env file is in git. Commit and push to GitHub.
- Vercel is already connected to the repo (no production deployment yet, because there was no app).
  This push triggers the first real deploy to https://sub-agents-three.vercel.app. Make sure `npm
run build` passes locally first.
- Once I confirm it's deployed, check the Vercel URL shows the search form.
- Give me a short summary and a list of anything ambiguous or not matched.
  Don't silently guess. Stop; don't start Step 2.
```

4. Review gate

```
/step-check 1
```

5. Step 2: HasData data layer

Raw API clients, one live call per source saved as fixtures, normalizers, product rules, vitest, dev route.

```
## Context
Sub-Agents project. Step 1 is done and passed /step-check.
This is STEP 2 ONLY (CLAUDE.md section 13): the HasData data layer.
No Anthropic SDK, no agents, no UI changes yet. Re-read CLAUDE.md sections 3, 4 and 6 first.

## Credit rule (important)
HasData calls cost credits. Make EXACTLY ONE live call per source in this step, to capture
real responses. After that, everything (tests, dev route) runs on saved fixtures.
Sample search for the live calls: destination "Goa", check-in 14 days from today,
1 night, 2 adults, 1 child age 5.

## Tasks
```

```
### 1. Raw API clients
- src/lib/hasdata/booking.ts: GET https://api.hasdata.com/scrape/booking/search
  with keyword, checkInDate, checkOutDate, rooms=1, adults, children (with ages), INR currency.
- src/lib/hasdata/airbnb.ts: GET https://api.hasdata.com/scrape/airbnb/listing
  with location, checkIn, checkOut, adults, children, INR if supported.
- Header x-api-key from HASDATA_API_KEY (server-side only). Clear errors for a missing key,
  401, non-200 and timeouts (15s).
- Check the HasData docs for each endpoint's exact param names and how child ages are passed.
  Don't guess param names.

### 2. Capture real responses
- Make the one live call per source. Save to fixtures/booking-raw.json and fixtures/airbnb-raw.json.
- Write docs/hasdata-fields.md: which raw fields map to each Stay field, per source, and
  whether the total price includes taxes and fees.
- Confirm whether Airbnb supports INR. If not, tell me and stop before converting currencies.

### 3. Normalize + product rules
- normalizeBooking(raw) and normalizeAirbnb(raw) → Stay[] (exact shape from CLAUDE.md section 6).
- Airbnb rating × 2 to /10. Set feesMayBeExtra for Airbnb when the total may exclude fees.
- src/lib/rules.ts applyRules(stays) → top 3: rating 7.0+/10, unrated excluded, no
  hostels/dorms/shared rooms, total incl. taxes & fees in INR, 1 room, unparseable price
  skipped, 3 cheapest.
- Add vitest. Unit tests for normalize and rules using the fixtures.

### 4. Dev test route
- GET /api/dev/stays?source=booking|airbnb returns the normalized top 3 from fixtures.
- Add &live=1 to call the real API. Return 404 for this route in production.

## Done when
- Both fixtures saved, field map written, tests pass, dev route returns 3 valid stays per source.
- Commit and push to GitHub (Vercel redeploys automatically).
- Summarize, including: how many raw listings each source returned, how many passed the rules, and
anything ambiguous in the field mapping. Stop; don't start Step 3.
```

6. Decisions after the Airbnb-currency finding (follow-up to Step 2)

Airbnb returned USD with no currency option and no total, so Step 2 stopped and asked. You answered:

```
Go with the simple route:
1. Airbnb currency: Fixed rate. Put USD_TO_INR as one constant in src/lib/config.ts, tell me
   the value you used, and mark Airbnb prices as approximate (feesMayBeExtra = true).
2. Airbnb total: use the price breakdown ("N nights x $X") to get the stay total
   (nights × nightly price), then convert to INR.
3. Booking total: pricePerStayParsed + excludedChargesParsed (incl. taxes & fees). Use
   reviews.score as the /10 rating, not the star class.
4. Airbnb child ages: not supported, so just pass the children count. That's fine.
```

```
No extra live calls. Continue with normalize, rules, tests, field-map doc and dev route.
I've updated CLAUDE.md with these decisions; include it in this step's commit.
```

7. Review gate

```
/step-check 2
```

8. Step 3: The two subagents

What you typed:

```
Build Step 3: follow the "Step 3: The two subagents" prompt in docs/build-prompts.md exactly.
At the end, leave the dev server running so I can open the dev route in my browser.
```

The prompt it points to (verbatim from the file):

```
## Context
Sub-Agents project. Steps 1–2 done and passed /step-check.
This is STEP 3 ONLY (CLAUDE.md section 13): the Booking and Airbnb subagents.
No orchestrator, no UI changes yet. Re-read CLAUDE.md section 3 first.

## Tasks

### 1. Shared Anthropic client
- Install @anthropic-ai/sdk.
- src/lib/anthropic.ts: the ONLY place the client is created, using ANTHROPIC_API_KEY and
  ANTHROPIC_MODEL. Throw a clear error if either is missing. Server-side only.

### 2. Fixture switch
- USE_FIXTURES is already in .env.example (Step 1) and set to true in my .env.local.
  When true, the search tools return the Step 2 fixtures instead of calling HasData.
  Treat a missing value as true locally.

### 3. Subagents
- src/lib/agents/bookingAgent.ts and airbnbAgent.ts, each exporting
  runBookingAgent(input: SearchInput) / runAirbnbAgent(input).
- Each is its own Claude Messages API call with:
  - its own short system prompt (role: "You find stays on {source}. Use your tool once.
    Don't rank or filter, just report what the tool returned.")
  - exactly ONE tool: search_booking / search_airbnb, with a JSON schema for the search params.
- Tool-use loop: the model calls the tool → our code runs fetch → normalize → applyRules →
  the result goes back to the model as the tool_result → the model replies with one short line.
- IMPORTANT: the returned stays come from our tool code, never parsed from the model's text.
- Return { stays: Stay[], ms, trace } where trace logs: agent started, tool called (params),
  tool returned (count), agent finished.
- Cap the loop (max 3 turns). Errors become a failed SourceResult, not a crash.
```

```
### 4. Dev route
- GET /api/dev/agent?source=booking|airbnb runs one subagent on a sample input
  and returns { stays, ms, trace }. 404 in production.

## Done when
- Both subagents run in fixture mode and return 3 stays plus a readable trace.
- One live run of each in Anthropic (USE_FIXTURES=true, so no HasData credits) works.
- Commit and push to GitHub (Vercel redeploys automatically). Summarize, with an example trace.
Stop; don't start Step 4.
```

9. Follow-up: subagents correct the model's tool params

```
Small change to the subagents: after the model calls its tool, compare its params with the
user's SearchInput. If anything differs (destination, dates, adults, children, ages), use the
SearchInput values and add a "params corrected" warning line to the trace. Add a test for it.
Commit and push.
```

10. Review gate

```
/step-check 3
```

11. Step 4: Orchestrator, rank() and streaming

What you typed:

```
Build Step 4: follow the "Step 4: Orchestrator, rank() and streaming" prompt in docs/build-
prompts.md exactly (I just updated it, so re-read the file and include it in this step's commit).
```

The prompt it points to (verbatim from the file):

```
## Context
Sub-Agents project. Steps 1–3 done and passed /step-check.
This is STEP 4 ONLY (CLAUDE.md section 13): orchestrator, rank(), Best pick reason, SSE.
No UI changes yet. Re-read CLAUDE.md sections 3, 5, 6 and 7 first.

## Tasks

### 1. rank()
- src/lib/rank.ts, a pure function exactly per CLAUDE.md section 5:
  sort by totalPrice asc (equal price: higher rating first), then repeatedly swap adjacent
  A,B when B is at most 10% pricier AND rated at least 1.0 higher, until no swaps.
- Vitest tests: no swaps, one swap, chain swaps, ties, single stay, empty list,
  exactly 10% / exactly 1.0 boundaries.

### 2. Orchestrator
- src/lib/agents/orchestrator.ts, runOrchestrator(input, emit).
```

```
- Its own Claude call with a system prompt ("You coordinate two subagents. Call both tools in
  the same turn.") and exactly 2 tools: call_booking_agent, call_airbnb_agent.
- The tool handlers call the existing runBookingAgent / runAirbnbAgent (Step 3). Each already
  returns a SourceResult plus its own trace; merge those traces into the orchestrator trace
  in time order, tagged by agent.
- When the model returns the tool_use blocks, run both subagents IN PARALLEL with
  Promise.allSettled. If the model only calls one, call the other in code and log a warning
  in the trace.
- The subagents' input always comes from the user's SearchInput (same rule as Step 3), not
  from what the orchestrator model writes in its tool calls.
- Return both results to the model as tool_results.
- Code combines the stays → rank() → max 6.
- Then the model writes the Best pick reason only: give it the #1 and #2 stays, ask for one line,
  max ~90 chars, comparing price and rating (e.g. "Only ₹350 more than the cheapest, but rated
  9.2 vs 7.4."). If it's longer than 90 chars or empty, fall back to a code-written line.
- Fewer than 2 stays: reason is null (1 stay) or no Best pick (0 stays).
- Build SearchResult { stays, bestPickReason, sources, trace }.

### 3. POST /api/search (SSE)
- Validate the body with validateSearchInput() (400 on bad input).
- Stream events: source_started, source_done {count}, source_failed {error},
  ranking_started, result {SearchResult}. Emit source events as each subagent actually
  starts and finishes.
- Node runtime, export maxDuration = 60.
- One subagent failing → partial result. Both failing → result with empty stays and both failed.

### 4. Browser view for learning
- GET /api/dev/search?fail=booking|airbnb (fail is optional) runs the orchestrator on a sample
  input (Goa, 14 days from today, 1 night, 2 adults, 1 child age 5) and returns JSON:
  { events: [every SSE event in order, with time], result: SearchResult, lines: [readable trace] }.
  The optional fail param forces that subagent to fail, to show the partial state.
  404 in production.

## Done when
- rank() tests pass.
- A curl POST to /api/search in fixture mode streams all events in order and ends with a valid
  result (show me the output).
- Forcing one source to fail (e.g. a temporary env flag) still returns the other's stays.
- Commit and push to GitHub (Vercel redeploys automatically). Summarize, with the full trace of one
run.
- Leave the dev server running so I can open /api/dev/search in my browser. Stop; don't start Step
5.
```

12. Follow-up: honest Best pick reason

```
Tighten the Best pick reason prompt: be honest about the trade-off. If #1 is cheaper but rated
lower, say so (e.g. "₹663 cheaper than the next option, rated 9.5 vs 10."). If #1 is pricier
```

```
but rated higher, say that (e.g. "Only ₹350 more than the cheapest, but rated 9.2 vs 7.4.").
Never call a lower rating a plus. Add a test for each case. Commit and push.
```

13. Review gate

```
/step-check 4
```

14. Step 5: Loading, Results and all states

What you typed:

```
Build Step 5: follow the "Step 5: Loading, Results and all states" prompt in docs/build-prompts.md
exactly.
Also: show ratings with one decimal (9.52 → "9.5 / 10", 10 → "10 / 10").
At the end, leave the dev server running so I can try the full app at localhost:3000.
```

The prompt it points to (verbatim from the file):

```
## Context
Sub-Agents project. Steps 1–4 done and passed /step-check.
This is STEP 5 ONLY (CLAUDE.md section 13): the results experience.
Re-read docs/design-spec-v3.md Screens 2–4 and match copy, sizes and colors exactly.

## Tasks

### 1. Connect the search
- On Search submit: button shows "Searching…" (disabled, opacity 0.6), then go to /results.
- /results reads the params, POSTs to /api/search and reads the SSE stream with fetch +
  ReadableStream (EventSource can't POST).

### 2. Loading (Screen 2)
- "Finding your stays…" + summary line.
- Two rows with 16px #2563eb spinners: "Searching Booking.com…" / "Searching Airbnb…".
- Update each row live: "✓ Booking.com: {n} stays found", or muted "✗ Booking.com: unavailable".
- Then "Picking the best stay…" on ranking_started.

### 3. Results (Screen 3)
- Header: "Best stays for your trip", summary line
  "{destination} · {dates} · {n} night(s) · {guests}", sub-line
  "{n} stays compared from Booking.com and Airbnb", link "← New search" (keeps the inputs).
- Cards per spec: source pill, name, price "₹4,200 total stay", "8.2 / 10 rating",
  "View on Booking.com" / "View on Airbnb" (new tab), "Fees may be extra" when flagged.
- Best pick card (first only): 2px #2563eb border, #eff6ff background, "⭐ Best pick" pill
  before the source pill, reason line at the bottom.

### 4. Other states (Screen 4), exact copy
- Partial, Empty, Error, and 1–5 stays.
```

```
### 5. Agent activity (learning view)
- Not in the design spec, so hide it by default. Show a small "Agent activity" timeline under
  the results only when the URL has ?debug=1, using the trace.

## Done when
- A full search works in fixture mode: loading rows update live, results and Best pick render.
- Each state checked (force failures/empties in fixture mode). Desktop and <600px look right.
- Commit and push to GitHub (Vercel redeploys automatically). Summarize, with anything that doesn't
match the spec. Stop; don't start Step 6.
```

15. Follow-up: two small fixes

```
Two small fixes:
1. Partial sub-line: name only the source(s) that returned results, e.g. "3 stays compared
   from Booking.com". Keep "from Booking.com and Airbnb" when both succeeded.
2. Server-side check-in validation: allow check-in up to 1 day before the server's UTC date,
   so visitors west of UTC aren't wrongly told "check-in can't be in the past". Client-side
   validation stays as is. Add a test.
Commit and push, keep the dev server running.
```

16. Review gate

```
/step-check 5
```

17. Step 6: Go live with real data

What you typed:

```
Build Step 6: follow the "Step 6: Go live with real data" prompt in docs/build-prompts.md exactly.
I've confirmed in Vercel: USE_FIXTURES=false and the Anthropic keys are set.
Run only ONE live search on https://sub-agents-three.vercel.app for the final check, and tell me how
long it took.
```

The prompt it points to (verbatim from the file):

```
## Context
Sub-Agents project. Steps 1–5 done and passed /step-check.
This is STEP 6 ONLY (CLAUDE.md section 13): ship it.

## Tasks

### 1. Pre-ship checks
- npm run build, lint and tests pass.
- Confirm no env file or key anywhere in git history (git log -p search). If found, stop and tell
me.
```

```
### 2. README.md
- What it is, how to run locally, env vars (names only).
- How it works: the orchestrator, how the 2 subagents are invoked in parallel, where code
  (rules, rank) takes over from the LLM. Include the agent flow diagram from CLAUDE.md.
- Fixture mode vs live mode, and the credit cost per live search.

### 3. Go live with real data
- GitHub and Vercel are already connected (Steps 0–1). Commit and push main.
- USE_FIXTURES is already false on Vercel. Ask me to confirm it in Vercel settings; never show
values.
- Confirm the /api/search route doesn't time out on Vercel.

## Done when
- Live URL works end to end with real data: one full search, Best pick shown.
- Dev routes return 404 in production.
- Give me: the live URL, the repo URL, and the credits one live search used. Done.
```

18. Reply to "run one more live search?"

```
No, I'll test it myself in the browser.
```

19. Bug reports from your own testing (these led to two real fixes)

a) Airbnb always "unavailable":

```
Why am I always seeing "Airbnb unavailable"?
```

b) With screenshots of Madurai (Booking only, "Airbnb results are unavailable right now"):

```
I tried Madurai, Chennai, Delhi and Kanchipuram. For all of them Airbnb is always unavailable.
If there are any rules that are failing every time, let's see if we can take that off.
```

→ Root cause found: HasData's Airbnb endpoint rejects `children=0` (HTTP 422). Fix: omit `children` when there are none.

c) With a screenshot of Chennai, 38 nights (Booking only, Airbnb gave no stays):

```
It is working, but I tried a weird case: 38 nights, and Airbnb did not return rooms.
```

→ Root cause found: stays of about 28+ nights are priced monthly ("Average monthly price $M"), which the parser didn't understand. Fix: price = monthly ÷ 30 × nights.

20. Wrap-up and this request

```
Works, let's stop here.
```

> I think this project is fetching the top 3 hotels from Booking and Airbnb, and also fetching UI design from Figma.
> I want to relearn. Can you consolidate all valid prompts in sequence, end to end, without missing anything?
> See all the chat in this project and come up with consolidated prompts.

Part C. Not prompts (left out on purpose)

- The pasted log `airbnb agent started → called search_airbnb … ← tool returned 3 stays …`. It was a trace example, not an instruction, so it was asked about, not acted on.
- Automated system notices (background task finished, MCP servers connecting, instruction files re-read).
- `/step-check` appears above as the review gate each time; its definition lives in `.claude/skills/step-check/SKILL.md`.

Part D. Gotchas found while building (fold these into the prompts if you rebuild)

Where	What happened	Put this in the prompt next time
Step 2, Booking	`currency=INR` is rejected (HTTP 422); the enum is lowercase. Child ages go in `childrenAges`.	"Booking: `currency=inr`, ages in `childrenAges` (comma-separated)."
Step 2, Airbnb	Prices are USD (no currency param), no child-age param, no total field.	Already added to CLAUDE.md: fixed `USD_TO_INR`, nights × nightly.
After Step 6, Airbnb	`children=0` is rejected (HTTP 422, must be ≥ 1).	"Airbnb: send `children` only when > 0."
After Step 6, Airbnb	Stays of about 28+ nights use monthly pricing ("Average monthly price").	"Airbnb long stays: monthly price ÷ 30 × nights; get nights from the echoed search URL."
Step 1, Next 16	`new Date()` in a client component fails prerender.	"Derive today's date client-side (`useSyncExternalStore`), not at render."
Step 2, tooling	vitest wouldn't install with `@types/node@^20`.	"Use `@types/node@^24`."
Step 3, lint	A function named `useFixtures` trips the React hooks rule.	"Name it `fixturesEnabled()`."
Step 4, Next 16	`export const runtime` is not allowed with `cacheComponents`.	"Keep `maxDuration = 60`; skip the `runtime` export."
Step 5, UI	Next keeps the Search page alive, so "Searching…" stayed disabled after "← New search".	"Reset the button state when leaving the page."
Step 6, Vercel	Reading `searchParams` on the server made `/results` blank for about 30 s in production (pre-rendered shell).	"Read the URL params in the browser (`useSearchParams` inside Suspense)."
Step 5/6, time zones	Server (UTC) can reject a visitor's "today".	"Server accepts check-in from 1 day before its UTC date."

Where	What happened	Put this in the prompt next time
Step 4, reason	The model could call a lower rating a plus.	"Be honest about the price/rating trade-off; give the model code-computed facts."